SIMATS ENGINEERING

ASSESSMENT TOOL 1: Experiments

COURSE NAME: COMPUTER VISION **COURSE CODE: ITA05**

COURSE OUTCOME COVERED

CO3: *Analyze and extract image features using detection and matching techniques for effective image representation and comparison.* (BTL 4)

Assessment Tool Weightage: 40%

Code of Conduct:

I G. Manish / 192210591 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: G. Manish

RUBRICS FOR CALCULATION: 90/10

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation	Correct implementation with	Basic implementation with errors or	Incorrect or incomplete implementation

1. Preprocessing Impact Analysis

Design and perform an experiment to study the impact of preprocessing techniques such as noise removal and normalization on image quality and feature detection. Explain the theoretical concepts behind preprocessing methods and justify the selection of techniques used. Implement the preprocessing process using suitable software/tools, document the workflow and outputs systematically, and analyze how preprocessing influences feature detection accuracy and reliability.

2. Image Representation Study

Conduct an experiment to convert a given image into grayscale and binary representations. Explain the significance of different image representations in computer vision applications. Design the conversion procedure using appropriate tools, document the generated outputs clearly, and analyze how image representation affects edge detection and feature extraction performance.

3. Edge Detection Comparison

Design an experiment to compare Sobel and Canny edge detection techniques on the same input image. Explain the underlying concepts and operational principles of both methods. Apply the techniques using suitable tools/software, document the detected edges systematically, and analyze the comparative effectiveness of each method for different image conditions and applications.

4. Harris Corner Detection Analysis

Perform Harris corner detection on images containing varying textures and patterns. Explain the concept of corner detection and its importance in computer vision tasks. Use appropriate parameters and tools to execute the experiment, document the detected corner points clearly, and analyze the distribution, accuracy, and significance of the detected corners.

5. Effect of Noise on Corner Detection

Design an experiment to study the effect of noise on Harris corner detection performance. Explain the purpose of introducing noise and its influence on image features. Add noise systematically, perform corner detection using suitable tools, document observations for original and noisy images, and analyze the impact of noise along with possible improvement techniques.

6. Hough Transform for Shape Detection

Conduct an experiment using the Hough Transform to detect lines or circles in an image. Explain the principles and significance of the Hough Transform in shape detection. Execute the detection process using appropriate tools, record the detected shapes and their parameters, and analyze the detection accuracy under different image conditions.

7. Feature Matching using SIFT

Perform feature extraction and matching between two images using the SIFT algorithm. Explain the concept of feature matching and scale-invariant features. Use suitable software/tools to extract and match features, document the matching key points systematically, and analyze the robustness of SIFT under variations in scale, rotation, and illumination.

8. PCA for Feature Reduction

Design and implement an experiment using Principal Component Analysis (PCA) for dimensionality reduction of extracted image features. Explain the importance of feature reduction in machine learning and computer vision applications. Apply PCA using appropriate tools, document the feature dimensions before and after reduction, and analyze its impact on computational efficiency and feature representation quality.

9. Preprocessing vs Feature Extraction

Conduct a comparative study of feature extraction performed with and without preprocessing. Explain the relationship between preprocessing and feature extraction performance. Execute both approaches using suitable tools, document the observations systematically, and analyze the differences in feature quality, detection accuracy, and overall system performance. 10. Integrated Feature Detection Pipeline

Design and implement a complete image processing pipeline consisting of preprocessing, edge detection, and feature extraction stages. Explain the purpose and workflow of the integrated pipeline. Execute each stage systematically using appropriate tools, document intermediate and final outputs clearly, and analyze the contribution and importance of each stage in achieving accurate feature detection results.

11. Effect of Smoothing on Edge Detection

Design and perform an experiment to study the impact of smoothing techniques on edge detection performance. Explain the theoretical purpose of smoothing in reducing noise and improving edge continuity. Apply smoothing and edge detection methods using appropriate tools, document outputs before and after smoothing systematically, and analyze how smoothing affects edge clarity, continuity, and noise suppression.

12. Gradient-Based Edge Detection Study

Conduct an experiment using gradient-based edge detection methods to identify image boundaries. Explain the principles of gradient operators and their role in edge

detection. Implement the method using suitable tools/software, record gradient values and detected edges systematically, and analyze edge strength, accuracy, and sensitivity to image variations.

13. Prewitt vs Sobel Comparison

Design a comparative study of Prewitt and Sobel edge detection operators. Explain the conceptual differences and applications of both methods. Execute both operators accurately on the same image using appropriate tools, document the outputs clearly, and analyze their performance in detecting edges under varying image conditions.

14. Effect of Image Scaling on Feature Detection

Perform an experiment to analyze how image scaling affects feature detection accuracy. Explain the importance of scale in computer vision and feature extraction tasks. Resize images to different scales, apply feature detection techniques, document detected features systematically, and analyze the impact of scaling on feature consistency and detection performance.

15. Rotation Impact on Feature Matching

Conduct an experiment to evaluate the effect of image rotation on feature matching accuracy. Explain the significance of rotation invariance in feature matching algorithms. Rotate images at different angles, perform feature matching using suitable tools, document matching results clearly, and analyze the robustness of the method against rotational transformations.

16. Threshold Effect in Edge Detection

Design an experiment to study the influence of threshold values on edge detection quality. Explain the importance of thresholding in distinguishing meaningful edges from noise. Apply edge detection using different threshold values, record the outputs systematically, and analyze how threshold selection affects edge continuity, accuracy, and noise sensitivity.

17. Noise Removal Comparison

Perform a comparative study of different noise removal techniques applied to images. Explain the objectives and principles of commonly used filtering methods. Implement multiple noise reduction filters using suitable tools, document the outputs clearly, and analyze the effectiveness of each technique in preserving image details while removing noise.

18. Feature Detection in Noisy Images

Conduct an experiment to analyze feature detection performance on noisy images. Explain how image noise affects feature extraction and detection accuracy. Apply feature detection techniques on noisy images using appropriate tools, record detected features systematically, and analyze the impact of noise on feature reliability and stability.

19. Edge Detection under Illumination Changes

Design an experiment to study the effect of illumination variation on edge detection performance. Explain the relationship between lighting conditions and edge extraction accuracy. Apply edge detection methods on images with varying illumination levels, document the outputs clearly, and analyze the robustness of the techniques under different lighting conditions.

20. Corner Detection Parameter Study

Perform a parameter analysis study for Harris corner detection. Explain the significance of parameters in influencing corner detection accuracy. Execute the corner detection algorithm using different parameter values, record the detected corners systematically, and analyze how parameter variations affect detection sensitivity and performance.

21. Hough Transform Parameter Analysis

Conduct an experiment to evaluate the impact of parameter variations in the Hough Transform. Explain the role of parameters in shape detection accuracy. Apply the Hough Transform using different parameter settings, document the detected shapes and outputs clearly, and analyze the sensitivity and reliability of the detection process.

22. Feature Matching Accuracy Evaluation

Design an experiment to evaluate the accuracy of feature matching between images. Explain the objectives and significance of feature matching evaluation. Perform feature matching using suitable algorithms and tools, record matching points systematically, and analyze matching accuracy, false matches, and overall reliability.

23. PCA Component Selection Study

Perform a study on the effect of varying the number of principal components in PCA. Explain the importance of component selection in dimensionality reduction. Execute PCA with different numbers of components, document feature dimensions and outputs clearly, and analyze the trade-off between dimensionality reduction and information preservation.

24. Edge Detection vs Corner Detection

Conduct a comparative analysis of edge detection and corner detection techniques on the same image. Explain the objectives and differences between both approaches. Apply suitable methods using appropriate tools, document the detected features systematically, and analyze the types of features captured and their relevance in computer vision applications.

25. Multi-Stage Pipeline Analysis

Design and implement a multi-stage image processing pipeline consisting of preprocessing, edge detection, and feature extraction. Explain the objectives and workflow of the pipeline. Execute each stage systematically using suitable tools, document outputs at every stage clearly, and analyze the contribution and effectiveness of each stage in improving overall system performance.

26. Feature Detection Density Analysis

Perform an experiment to analyze the density of detected image features and its impact on performance. Explain the importance of feature density in computer vision tasks. Execute feature detection using appropriate tools, record the number and distribution of detected features systematically, and analyze the relationship between feature density, computational complexity, and detection accuracy.

27. Effect of Blurring on Feature Detection

Conduct an experiment to study the influence of image blurring on feature detection. Explain the purpose of blurring and its effect on image details. Apply blur filters followed by feature detection techniques, document the outputs clearly, and analyze how blurring affects feature visibility, accuracy, and robustness.

28. PCA Visualization Study

Perform a visualization study of features before and after applying PCA. Explain the significance of dimensionality reduction in feature analysis. Execute PCA using suitable tools, document visual outputs systematically, and analyze how PCA transforms and compresses feature representations while preserving important information.

29. Feature Matching under Noise and Rotation

Design an experiment to evaluate feature matching performance under combined noise and rotation conditions. Explain the challenges introduced by image transformations and noise. Apply noise and rotational transformations, perform feature matching using suitable tools, document the matching results clearly, and analyze the robustness and reliability of the matching technique.

30. End-to-End Vision System Experiment

Design and implement a complete computer vision pipeline involving preprocessing, feature detection, feature matching, and PCA-based dimensionality reduction. Explain the objectives and workflow of the integrated system. Execute all stages systematically using appropriate tools, document outputs at each stage clearly, and analyze the overall system performance, limitations, and bottlenecks.

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of	Appropriate use of	Limited or inconsistent	Incorrect or minimal use

		tools/technologies with proper configuration	**tools with correct execution**	**use of tools**	**of tools**
Analysis of Results	**15**	**Insightful analysis with accurate interpretation and validation of results**	**Good analysis with reasonable interpretation**	**Basic analysis with limited insights**	**Poor or incorrect analysis of results**
Documentation	**10**	**Clear, wellstructured documentation with diagrams, code, and results**	**Organized documentation with necessary details**	**Basic documentation with missing details**	**Poor or incomplete documentation**

1. Preprocessing Impact Analysis

Aim: To study how noise removal and normalization affect image quality and feature detection.

Theory: Preprocessing improves an image before feature extraction. Noise removal reduces unwanted pixels, while normalization improves the intensity distribution.

Procedure:

1. Read the image.
2. Convert it to grayscale.
3. Add or use a noisy image.
4. Apply Gaussian/median filtering.
5. Normalize the image.
6. Perform feature detection.
7. Compare original and preprocessed results.

Code:

import cv2 import matplotlib.pyplot as plt img = cv2.imread("image.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY) filtered =

cv2.GaussianBlur(gray, (5,5), 0) normalized = cv2.normalize(filtered,
None, 0, 255, cv2.NORM_MINMAX) edges_original = cv2.Canny(gray,
100, 200) edges_processed = cv2.Canny(normalized, 100, 200)
plt.figure(figsize=(10,6)) plt.subplot(2,2,1) plt.imshow(gray, cmap="gray")
plt.title("Original") plt.subplot(2,2,2) plt.imshow(normalized,
cmap="gray") plt.title("Preprocessed") plt.subplot(2,2,3)
plt.imshow(edges_original, cmap="gray") plt.title("Original Edges")
plt.subplot(2,2,4)
plt.imshow(edges_processed, cmap="gray")
plt.title("Processed Edges") plt.show()

Observation: Preprocessing reduces noise and produces clearer edges.

Result: Preprocessing improves feature detection accuracy and reliability.

2. Image Representation Study

Aim: To convert an image into grayscale and binary representations and study their effect on feature extraction.

Theory: A grayscale image contains intensity information, while a binary image contains only two values: 0 and 255.

Procedure:

1. Read the image.
2. Convert it to grayscale.
3. Apply binary thresholding.
4. Compare the representations. import cv2 import matplotlib.pyplot as plt img = cv2.imread("image.jpg") gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

_, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY) plt.subplot(1,3,1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original") plt.subplot(1,3,2) plt.imshow(gray,
cmap="gray") plt.title("Grayscale") plt.subplot(1,3,3)
plt.imshow(binary, cmap="gray") plt.title("Binary")
plt.show()

Observation: Grayscale preserves intensity information, whereas binary representation simplifies the image.

Result: Grayscale is useful for edge and feature extraction, while binary images are useful for segmentation and shape analysis.

3. Edge Detection Comparison – Sobel vs Canny

Aim: To compare Sobel and Canny edge detection.

Theory: Sobel calculates image gradients in horizontal and vertical directions. Canny uses smoothing, gradient calculation, non-maximum suppression and hysteresis thresholding.

```
import cv2 import numpy as np import matplotlib.pyplot as plt img =
cv2.imread("image.jpg", 0) sobelx = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3) sobely =
cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3) sobel =
cv2.magnitude(sobelx.astype(np.float32), sobely.astype(np.float32)) canny =
cv2.Canny(img, 100, 200) plt.subplot(1,2,1) plt.imshow(sobel, cmap="gray")
plt.title("Sobel") plt.subplot(1,2,2) plt.imshow(canny, cmap="gray") plt.title("Canny")
plt.show()
```

Observation: Sobel produces gradient information, while Canny generally produces thinner and cleaner edges.

Result: Canny is more effective when accurate edge detection is required.

4. Harris Corner Detection Analysis

Aim: To detect corners using the Harris corner detection method.

Theory: A corner is a point where intensity changes significantly in more than one direction. Harris detects such points using the variation of intensity around a pixel.

```
import cv2 import numpy as
np import matplotlib.pyplot as
plt img =
cv2.imread("image.jpg") gray
= cv2.cvtColor(img,
cv2.COLOR_BGR2GRAY)
gray = np.float32(gray) dst =
cv2.cornerHarris(gray, 2, 3,
0.04) dst = cv2.dilate(dst,
None) result =
```

cv2.imread("image.jpg")
result[dst > 0.01 * dst.max()] =
[0, 0, 255]
plt.imshow(cv2.cvtColor(result
, cv2.COLOR_BGR2RGB))
plt.title("Harris Corners")
plt.show()

Observation: Corners are detected mainly at points where two or more edges meet.

Result: Harris successfully identifies important corner features.

5. Effect of Noise on Corner Detection

Aim: To study the effect of noise on Harris corner detection.

Theory: Noise introduces unwanted intensity variations, which may create false corner responses. import cv2 import numpy as np import matplotlib.pyplot as plt img = cv2.imread("image.jpg", 0) noise = np.random.normal(0, 25, img.shape) noisy = np.clip(img + noise, 0, 255).astype(np.uint8) def harris(image): f = np.float32(image) dst = cv2.cornerHarris(f, 2, 3, 0.04) return cv2.dilate(dst, None) original_corners = harris(img) noisy_corners = harris(noisy) plt.subplot(1,2,1) plt.imshow(original_corners, cmap="gray") plt.title("Original") plt.subplot(1,2,2) plt.imshow(noisy_corners, cmap="gray") plt.title("Noisy")
plt.show()

Observation: Noise produces additional unwanted intensity changes and can create false corners.

Result: Noise reduces the reliability of Harris corner detection. Preprocessing can improve performance.

6. Hough Transform for Shape Detection

Aim: To detect lines using the Hough Transform.

Theory: The Hough Transform converts edge points into parameter space and identifies shapes represented by common parameters.

import cv2 import numpy as np import
matplotlib.pyplot as plt img =
cv2.imread("image.jpg") gray = cv2.cvtColor(img,

```
cv2.COLOR_BGR2GRAY) edges = cv2.Canny(gray,
50, 150) lines = cv2.HoughLinesP( edges, 1, np.pi /
180, threshold=50, minLineLength=50,
maxLineGap=10
)
result = img.copy() if
lines is not None: for
line in lines:
x1, y1, x2, y2 = line[0] cv2.line(result, (x1,y1), (x2,y2),
(0,255,0), 2) plt.imshow(cv2.cvtColor(result,
cv2.COLOR_BGR2RGB)) plt.title("Detected Lines")
plt.show()
```

Observation: Straight lines in the image are identified.

Result: Hough Transform is effective for detecting geometric shapes such as lines.

7. Feature Matching using SIFT

Aim: To extract and match features between two images using SIFT.

Theory: SIFT detects features that are relatively invariant to scale and rotation.

```
import cv2 import matplotlib.pyplot as plt img1
= cv2.imread("image1.jpg", 0) img2 =
cv2.imread("image2.jpg", 0) sift =
cv2.SIFT_create() kp1, des1 =
sift.detectAndCompute(img1, None) kp2, des2
= sift.detectAndCompute(img2, None) bf =
cv2.BFMatcher() matches = bf.knnMatch(des1,
des2, k=2) good = [] for m, n in matches: if
m.distance < 0.7 * n.distance:
good.append(m) result = cv2.drawMatches(img1, kp1, img2, kp2,
good, None, flags=2) plt.imshow(result) plt.title("SIFT Feature
Matching") plt.show()
print("Good Matches:", len(good))
```

Observation: Corresponding feature points between the two images are connected.

Result: SIFT provides robust feature matching under changes in scale and rotation.

8. PCA for Feature Reduction

Aim: To reduce the dimensionality of extracted image features using PCA. **Theory:** PCA transforms high-dimensional data into a smaller number of principal components while retaining important information.

```
import numpy as np
from sklearn.decomposition import PCA X =
np.random.rand(100, 20) print("Original dimensions:",
X.shape) pca = PCA(n_components=5) X_reduced =
pca.fit_transform(X) print("Reduced dimensions:",
X_reduced.shape) print("Explained variance:",
pca.explained_variance_ratio_)
```

Observation: The number of features is reduced from 20 to 5.
Result: PCA reduces computational complexity while preserving major information.

9. Preprocessing vs Feature Extraction

Aim: To compare feature detection with and without preprocessing.
Theory: Preprocessing can suppress noise and improve the quality of subsequent feature extraction.

```
import cv2 img = cv2.imread("image.jpg", 0) edges_original =
cv2.Canny(img, 100, 200) filtered = cv2.GaussianBlur(img, (5,5), 0) edges_processed =
cv2.Canny(filtered, 100, 200) print("Original edge pixels:",
cv2.countNonZero(edges_original)) print("Processed edge pixels:",
cv2.countNonZero(edges_processed))
```

Observation: Preprocessing generally reduces false edges caused by noise.

Result: Preprocessing can improve feature quality and detection reliability.

10. Integrated Feature Detection Pipeline

Aim: To implement a complete preprocessing, edge detection and feature extraction pipeline.

Workflow: Input → Grayscale → Noise Removal → Edge Detection → Feature Extraction.

```
import cv2 img = cv2.imread("image.jpg") gray =
cv2.cvtColor(img, cv2.COLOR_BGR2GRAY) filtered =
```

cv2.GaussianBlur(gray, (5,5), 0) edges =

cv2.Canny(filtered, 100, 200) sift = cv2.SIFT_create()

keypoints, descriptors = sift.detectAndCompute(gray, None)

print("Number of keypoints:", len(keypoints))

Observation: Each stage improves the input for the next stage.

Result: The integrated pipeline successfully detects image features.

11. Effect of Smoothing on Edge Detection

Aim: To study how smoothing affects edge detection.

Theory: Smoothing reduces noise and can improve edge continuity, but excessive smoothing may remove fine details.

import cv2 import matplotlib.pyplot as

plt img = cv2.imread("image.jpg", 0)

edges1 = cv2.Canny(img, 100, 200)

smooth = cv2.GaussianBlur(img, (7,7), 0)

edges2 = cv2.Canny(smooth, 100, 200)

plt.subplot(1,2,1) plt.imshow(edges1,

cmap="gray") plt.title("Without

Smoothing") plt.subplot(1,2,2)

plt.imshow(edges2, cmap="gray")

plt.title("With Smoothing") plt.show()

Observation: Smoothing reduces noise but may remove some fine edges.

Result: Moderate smoothing improves edge continuity and reduces false edges.

12. Gradient-Based Edge Detection Study

Aim: To detect boundaries using gradient operators.

Theory: Gradient operators measure changes in image intensity. Large gradient values usually occur near boundaries.

import cv2 import numpy as np import

matplotlib.pyplot as plt img =

cv2.imread("image.jpg", 0) gx = cv2.Sobel(img,

cv2.CV_64F, 1, 0, ksize=3) gy = cv2.Sobel(img,

```
cv2.CV_64F, 0, 1, ksize=3) gradient =
cv2.magnitude( gx.astype(np.float32),
gy.astype(np.float32)
)
plt.imshow(gradient, cmap="gray")
plt.title("Gradient Magnitude") plt.show()
```

Observation: Large gradient values occur near strong boundaries.
Result: Gradient operators successfully identify image boundaries.

13. Prewitt vs Sobel Comparison

Aim: To compare Prewitt and Sobel operators.
Theory: Both detect intensity changes. Sobel uses weighted kernels and provides some smoothing, while Prewitt uses uniform weighting.

```
import cv2 import numpy as np
import matplotlib.pyplot as plt
img = cv2.imread("image.jpg", 0)
prewitt_x = cv2.filter2D( img, -1,
np.array([[-1,0,1],[-1,0,1],[-1,0,1]])
)
sobel_x = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
plt.subplot(1,2,1) plt.imshow(prewitt_x,
cmap="gray") plt.title("Prewitt") plt.subplot(1,2,2)
plt.imshow(np.abs(sobel_x), cmap="gray")
plt.title("Sobel") plt.show()
```

Observation: Both detect intensity changes. Sobel gives slightly more smoothing because of its kernel weighting.
Result: Sobel is generally more resistant to noise than Prewitt.

14. Effect of Image Scaling on Feature Detection

Aim: To study the effect of image scaling on feature detection.
Theory: Changing image scale can change the size and visibility of features. SIFT is designed to provide scale-invariant features. import cv2 img =
cv2.imread("image.jpg", 0) sift = cv2.SIFT_create() for scale in [0.5, 1.0, 1.5, 2.0]:

resized = cv2.resize(img, None, fx=scale, fy=scale) kp,
des = sift.detectAndCompute(resized, None)
print("Scale:", scale, "Features:", len(kp))
Observation: The number and distribution of detected features can change with image scale.
Result: Scale affects feature detection, while SIFT is designed to provide scale-invariant features.

15. Rotation Impact on Feature Matching

Aim: To study feature matching under image rotation.
Theory: Rotation invariance allows feature descriptors to match corresponding points even when the image orientation changes. import cv2 img = cv2.imread("image.jpg", 0) h, w = img.shape center = (w//2, h//2) matrix = cv2.getRotationMatrix2D(center, 45, 1.0) rotated = cv2.warpAffine(img, matrix, (w,h)) sift = cv2.SIFT_create() kp1, des1 = sift.detectAndCompute(img, None) kp2, des2 = sift.detectAndCompute(rotated, None) bf = cv2.BFMatcher() matches = bf.knnMatch(des1, des2, k=2) good = [m for m,n in matches if m.distance < 0.7*n.distance] print("Good matches:", len(good))
Observation: SIFT can maintain many corresponding features after rotation.
Result: SIFT demonstrates strong rotation robustness.

16. Threshold Effect in Edge Detection

Aim: To study the effect of threshold values on Canny edge detection.
Theory: Thresholds determine which gradient values are considered meaningful edges and influence noise sensitivity and edge continuity.
import cv2 import
matplotlib.pyplot as plt img =
cv2.imread("image.jpg", 0)
thresholds = [(50,100),
(100,200), (150,250)] for i,
(t1,t2) in enumerate(thresholds):
edges = cv2.Canny(img, t1, t2)
plt.subplot(1,3,i+1)

plt.imshow(edges, cmap="gray")
plt.title(f"{t1},{t2}") plt.show()
Observation: Low thresholds detect more edges but may include noise. High thresholds produce fewer, stronger edges.
Result: Proper threshold selection is important for accurate edge detection.

17. Noise Removal Comparison

Aim: To compare different noise removal filters.
Theory: Different filters are suited to different noise types. Median filtering is effective for impulse noise, Gaussian filtering provides smooth averaging, and bilateral filtering preserves edges better. import cv2 import matplotlib.pyplot as plt img = cv2.imread("image.jpg") median = cv2.medianBlur(img, 5) gaussian = cv2.GaussianBlur(img, (5,5), 0) bilateral = cv2.bilateralFilter(img, 9, 75, 75) plt.subplot(1,3,1)
plt.imshow(cv2.cvtColor(median, cv2.COLOR_BGR2RGB))
plt.title("Median") plt.subplot(1,3,2)
plt.imshow(cv2.cvtColor(gaussian, cv2.COLOR_BGR2RGB))
plt.title("Gaussian") plt.subplot(1,3,3)
plt.imshow(cv2.cvtColor(bilateral, cv2.COLOR_BGR2RGB))
plt.title("Bilateral") plt.show()
Observation: Median is effective for impulse noise, Gaussian provides smooth filtering, and bilateral filtering better preserves edges.
Result: The best filter depends on the type of noise and required edge preservation.

18. Feature Detection in Noisy Images

Aim: To analyze feature detection performance in noisy images.
Theory: Noise can introduce false intensity changes and reduce the stability of detected features. import cv2 import numpy as np img = cv2.imread("image.jpg", 0) noise = np.random.normal(0, 30, img.shape) noisy = np.clip(img + noise, 0, 255).astype(np.uint8) sift = cv2.SIFT_create() kp_original, _ = sift.detectAndCompute(img, None) kp_noisy, _ = sift.detectAndCompute(noisy, None) print("Original features:", len(kp_original)) print("Noisy features:", len(kp_noisy))
Observation: Noise can create false features and change the detected feature distribution.
Result: Noise reduces feature stability and reliability.

19. Edge Detection under Illumination Changes

Aim: To study the effect of illumination variation on edge detection.

Theory: Lighting changes can alter intensity differences and therefore change edge strength and detection results.

```
import cv2 img = cv2.imread("image.jpg", 0) dark =
cv2.convertScaleAbs(img, alpha=0.5, beta=0) bright =
cv2.convertScaleAbs(img, alpha=1.5, beta=30) edges_dark =
cv2.Canny(dark, 100, 200) edges_bright = cv2.Canny(bright,
100, 200) print("Dark edge pixels:",
cv2.countNonZero(edges_dark)) print("Bright edge pixels:",
cv2.countNonZero(edges_bright))
```

Observation: Illumination changes can modify edge strength and detection results.

Result: Proper normalization or illumination correction can improve robustness.

20. Corner Detection Parameter Study

Aim: To study how Harris parameters affect corner detection.

Theory: Harris parameters such as block size influence the neighborhood used to calculate corner response.

```
import cv2 import numpy as np img = cv2.imread("image.jpg", 0) gray =
np.float32(img) for block in [2, 3, 5]:
dst = cv2.cornerHarris(gray, block, 3, 0.04) count
= np.sum(dst > 0.01 * dst.max()) print("Block
size:", block, "Corners:", count)
```

Observation: Changing block size changes the number of detected corners.

Result: Harris parameters must be selected according to image characteristics.

21. Hough Transform Parameter Analysis

Aim: To study the effect of Hough Transform parameters.

Theory: Parameters such as accumulator threshold, minimum line length and maximum line gap control detection sensitivity.

```
import cv2 import numpy as np
img = cv2.imread("image.jpg", 0)
```

```
edges = cv2.Canny(img, 50, 150)
for threshold in [30, 50, 100]:
lines = cv2.HoughLinesP( edges,
1, np.pi/180,
threshold=threshold,
minLineLength=50,
maxLineGap=10
) count = 0 if lines is None else len(lines)
print("Threshold:", threshold, "Lines:", count)
```

Observation: Lower thresholds detect more lines, while higher thresholds detect only stronger lines.

Result: Hough parameters strongly affect shape detection sensitivity.

22. Feature Matching Accuracy Evaluation

Aim: To evaluate the accuracy of feature matching.

Theory: Good matches are those that satisfy a suitable descriptor-distance criterion. The ratio of good matches to total matches can be used as an approximate matching measure.

```
import cv2 img1 = cv2.imread("image1.jpg", 0)
img2 = cv2.imread("image2.jpg", 0) sift =
cv2.SIFT_create() kp1, des1 =
sift.detectAndCompute(img1, None) kp2, des2
= sift.detectAndCompute(img2, None) bf =
cv2.BFMatcher() matches = bf.knnMatch(des1,
des2, k=2) good = [] for m,n in matches: if
m.distance < 0.7*n.distance:
good.append(m) accuracy = len(good) /
max(len(matches), 1) * 100 print("Total
matches:", len(matches)) print("Good matches:",
len(good)) print("Matching ratio:", accuracy, "%")
```

Observation: Good matches represent reliable corresponding features.

Result: The matching ratio provides an approximate measure of matching reliability.

23. PCA Component Selection Study

Aim: To study how the number of PCA components affects information preservation.

Theory: Increasing the number of principal components generally preserves more variance but gives less dimensionality reduction. import numpy as np

from sklearn.decomposition import PCA

X = np.random.rand(100, 20) for n in [2,

5, 10, 15]:

pca = PCA(n_components=n)
pca.fit(X) variance =

sum(pca.explained_variance_ratio_)

print("Components:", n) print("Variance

retained:", variance)

Observation: Increasing the number of components preserves more information.

Result: There is a trade-off between dimensionality reduction and information preservation.

24. Edge Detection vs Corner Detection

Aim: To compare edge and corner detection.

Theory: Edge detection finds boundaries or locations of rapid intensity change. Corner detection finds points where intensity changes significantly in multiple directions.

import cv2 import numpy as np img =

cv2.imread("image.jpg", 0) edges =

cv2.Canny(img, 100, 200) gray =

np.float32(img) corners =

cv2.cornerHarris(gray, 2, 3, 0.04) print("Edge

pixels:", cv2.countNonZero(edges))

print("Corner points:", np.sum(corners > 0.01 * corners.max()))

Observation: Edges form continuous boundaries, while corners are individual interest points.

Result: Both methods capture different types of image features and are useful for different computer vision tasks.

25. Multi-Stage Pipeline Analysis

Aim: To implement preprocessing, edge detection and feature extraction as a multi-stage pipeline.

Workflow: Image → Grayscale → Filtering → Edge Detection → SIFT Feature Extraction.

```
import cv2 img = cv2.imread("image.jpg") gray =
cv2.cvtColor(img, cv2.COLOR_BGR2GRAY) filtered =
cv2.GaussianBlur(gray, (5,5), 0) edges =
cv2.Canny(filtered, 100, 200) sift = cv2.SIFT_create()
keypoints, descriptors = sift.detectAndCompute(gray, None)
print("Keypoints:", len(keypoints)) print("Descriptor size:",
descriptors.shape)
```

Observation: Each stage contributes to reliable image feature detection.

Result: The multi-stage pipeline improves overall feature analysis.

26. Feature Detection Density Analysis

Aim: To study the relationship between feature density and computational complexity.

Theory: More features can provide more image information, but they also increase matching and processing requirements.

```
import cv2 img = cv2.imread("image.jpg", 0) sift =
cv2.SIFT_create() keypoints, descriptors =
sift.detectAndCompute(img, None) height, width =
img.shape density = len(keypoints) / (height * width)
print("Number of features:", len(keypoints)) print("Feature
density:", density)
```

Observation: More features provide more information but increase computational requirements.

Result: Feature density should be balanced to achieve good accuracy without unnecessary computation.

27. Effect of Blurring on Feature Detection

Aim: To study how image blurring affects feature detection.

Theory: Blurring suppresses high-frequency details. As blur increases, fine structures may become less visible to feature detectors.

```
import cv2 img =
cv2.imread("image.jpg", 0) sift =
cv2.SIFT_create() for k in [3, 7,
11]:
blurred = cv2.GaussianBlur(img, (k,k), 0) kp,
des = sift.detectAndCompute(blurred, None)
print("Blur kernel:", k, "Features:", len(kp))
```

Observation: As blur increases, fine details disappear and the number of detectable features may decrease.

Result: Excessive blurring reduces feature visibility and detection accuracy.

28. PCA Visualization Study

Aim: To visualize features before and after PCA.

Theory: PCA projects high-dimensional features into a smaller coordinate system while retaining the directions of greatest variance.

```
import numpy as np import
matplotlib.pyplot as plt from
sklearn.decomposition import PCA X =
np.random.rand(100, 10) pca =
PCA(n_components=2) X_pca =
pca.fit_transform(X)
plt.scatter(X_pca[:,0], X_pca[:,1])
plt.xlabel("PC1") plt.ylabel("PC2")
plt.title("PCA Feature Visualization")
plt.show() print("Original dimensions:",
X.shape) print("PCA dimensions:",
X_pca.shape)
```

Observation: Ten-dimensional features are represented using two principal components.

Result: PCA provides a simpler visual representation while preserving major variations in the data.

29. Feature Matching under Noise and Rotation

Aim: To evaluate SIFT matching under combined noise and rotation.

Theory: Noise changes local image appearance, while rotation changes feature orientation. A robust descriptor should tolerate both to a reasonable degree.

```
import cv2 import numpy as np
img1 = cv2.imread("image.jpg", 0)
h, w = img1.shape
M = cv2.getRotationMatrix2D((w//2, h//2), 30, 1)
rotated = cv2.warpAffine(img1, M, (w,h)) noise =
np.random.normal(0, 20, rotated.shape) noisy_rotated =
np.clip(rotated + noise, 0, 255).astype(np.uint8) sift =
cv2.SIFT_create() kp1, des1 = sift.detectAndCompute(img1,
None) kp2, des2 = sift.detectAndCompute(noisy_rotated,
None) bf = cv2.BFMatcher() matches = bf.knnMatch(des1,
des2, k=2) good = [] for m,n in matches: if m.distance <
0.7*n.distance: good.append(m)
print("Good matches:", len(good))
```

Observation: Noise and rotation make matching more difficult, but SIFT can still identify many corresponding features.

Result: SIFT demonstrates robustness against moderate rotation and noise.

30. End-to-End Vision System Experiment

Aim: To implement a complete computer vision system involving preprocessing, feature detection, feature matching and PCA-based dimensionality reduction.

Workflow: Input Images → Preprocessing → SIFT Feature Detection → Feature Matching → PCA Dimensionality Reduction → Final Analysis.

```
import cv2 import numpy as np from
sklearn.decomposition import PCA
img1 = cv2.imread("image1.jpg", 0)
img2 = cv2.imread("image2.jpg", 0)
# Preprocessing img1 =
cv2.GaussianBlur(img1, (5,5), 0) img2
= cv2.GaussianBlur(img2, (5,5), 0)
```

```
# Feature detection sift = cv2.SIFT_create() kp1,
des1 = sift.detectAndCompute(img1, None)
kp2, des2 = sift.detectAndCompute(img2, None)
print("Image 1 features:", len(kp1))
print("Image 2 features:", len(kp2))
# Feature matching bf =
cv2.BFMatcher() matches =
bf.knnMatch(des1, des2, k=2) good = []
for m,n in matches: if m.distance <
0.7*n.distance: good.append(m)
print("Good matches:", len(good))
# PCA
if des1 is not None and len(des1) >= 2: n_components
= min(10, des1.shape[0], des1.shape[1]) pca =
PCA(n_components=n_components) reduced =
pca.fit_transform(des1) print("Original descriptor:",
des1.shape) print("Reduced descriptor:",
reduced.shape)
print("Variance retained:", sum(pca.explained_variance_ratio_))
```

Observation: The system performs preprocessing, extracts SIFT features, matches corresponding features, and reduces descriptor dimensions using PCA.

Result: The complete pipeline successfully demonstrates an end-to-end computer vision feature analysis system.